JAUME VILADOMS
Formació Professional integrada

Activitat 5

Les funcions

Escriure una vegada, fer servir moltes

Mòdul	M0613 · Desenvolupament web en entorn servidor
Cicle	CFGS Desenvolupament d'Aplicacions Web · 2n curs
Resultats d'aprenentatge	RA2 · RA3
Durada	3 hores
Curs	2026-2027

Índex

2
Presentació de l'activitat **3**
Els fitxers que has de crear 3
Apartat 1 · La primera funció **4**
Exercicis 4
Apartat 2 · Els paràmetres **4**
Exercicis 5
Apartat 3 · return no és echo **5**
Exercicis 6
Apartat 4 · Valors per defecte **6**
Exercicis 7
Apartat 5 · Els tipus i strict_types **7**
Exercicis 7
Apartat 6 · L'àmbit de les variables **8**
La paraula static 8
Exercicis 9
Apartat 7 · Funcions que treballen amb arrays **9**
Dos models 9
Exercicis 10
Repte final · El catàleg de BitBazar **10**
Funcions que ha de tenir 10
Què ha de mostrar la pàgina 11
Com s'entrega **11**

Presentació de l'activitat

Objectiu	RA2. Escriu sentències executables per un servidor web reconeixent i aplicant procediments d'integració del codi en llenguatge de marques. RA3. Escriu blocs de sentències embeguts en llenguatges de marques, seleccionant i utilitzant les estructures de programació.
Requisits	Navegador web, Visual Studio Code o editor de text i el servidor local (WAMP) en marxa.
Entrega	Lliurar un fitxer ZIP a Clickedu amb tots els documents generats.

Fins ara has fet servir funcions que ja venien fetes: count(), trim(), number_format(), filter_var()... En aquesta activitat n'escriuràs de teves.

Una funció és un tros de codi amb nom. L'escrius un cop i el crides les vegades que calgui. Serveix per no repetir codi, però sobretot serveix perquè, quan una cosa s'ha de canviar, només s'hagi de canviar en un lloc.

Els set apartats van de menys a més i cadascun és un fitxer independent. El repte final els ajunta tots en una sola pàgina.

> **La botiga:** continuem amb BitBazar, la botiga de material informàtic de l'activitat anterior. Pots canviar-li el nom i els productes si vols.

Els fitxers que has de crear

Crea una carpeta anomenada M0613_ACT05 dins del teu directori de treball del servidor i posa-hi aquests fitxers:

Fitxer	Què conté
`apartat1.php`	La primera funció: declarar i cridar
`apartat2.php`	Funcions amb paràmetres
`apartat3.php`	return contra echo
`apartat4.php`	Valors per defecte
`apartat5.php`	Els tipus i strict_types
`apartat6.php`	L'àmbit de les variables
`apartat7.php`	Funcions que treballen amb arrays
`repte.php`	El repte final
`estils.css`	El full d'estils, compartit per totes les pàgines

> **Important:** posa declare(strict_types=1); a la primera línia de tots els fitxers, just després de <?php. A l'apartat 5 veuràs per què.

Encara no toca: no has de fer servir include ni require. Si una funció et cal en dos fitxers, torna-la a escriure. Més endavant veurem com tenir-les en un sol lloc.

Apartat 1 · La primera funció

Una funció té quatre peces: la paraula function, un nom, uns paràmetres entre parèntesis i un cos entre claus. Declarar-la no l'executa: només existeix fins que algú la crida pel seu nom.

```
<?php
declare(strict_types=1);

/**
 * Mostra la capçalera de la botiga.
 *
 * @return void Només escriu per pantalla
 */
function capcalera(): void
{
    echo '<header><h1>BitBazar</h1></header>';
}

capcalera();   // ara si que s'executa
```

El tipus void vol dir que la funció no torna cap valor: només fa coses, com escriure per pantalla.

Exercicis

Crea el fitxer apartat1.php i fes-hi aquests exercicis:

Exercici 1

- Escriu la funció capcalera(), que mostri el nom de la botiga dins d'un <header> amb un <h1> i un subtítol.
- Escriu la funció peu(), que mostri un <footer> amb el nom de la botiga i l'any actual amb date('Y').
- Crida capcalera() una vegada al principi de la pàgina.
- Crida peu() tres vegades seguides al final i comprova que surt tres cops.
- Declara una tercera funció però NO la cridis. Comprova que no passa res i explica-ho en un comentari.

Per comprovar-ho: mira el codi font de la pàgina amb Ctrl+U. Hi han de sortir tres <footer> idèntics, però al teu codi el <footer> només està escrit una vegada.

Apartat 2 · Els paràmetres

Una funció que sempre fa exactament el mateix serveix de poc. Els paràmetres són les dades que li entren, i fan que la mateixa funció doni un resultat diferent cada vegada.

```
/**
 * Mostra una targeta de producte.
 *
 * @param string $nom   Nom del producte
```

```
 * @param float  $preu  Preu sense IVA
 * @param int    $estoc Unitats disponibles
 * @return void
 */
function targeta(string $nom, float $preu, int $estoc): void
{
    echo "<h3>$nom</h3>";
    echo "<p>$preu EUR · $estoc unitats</p>";
}

targeta('Teclat mecanic', 79.90, 4);
```

Sobre els paràmetres	Què vol dir
Van per ordre	El primer valor que passes va al primer paràmetre, el segon al segon...
Porten tipus	string, int, float, bool o array, davant del nom
Són variables locals	Només existeixen dins de la funció
Com menys, millor	Si en necessites més de quatre o cinc, la funció fa massa coses

Exercicis

Crea el fitxer apartat2.php i fes-hi aquests exercicis:

Exercici 1

- Escriu saluda(string $nom): void, que mostri un missatge de benvinguda amb el nom.
- Crida-la tres vegades amb tres noms diferents.

Exercici 2

- Escriu targeta(string $nom, float $preu, int $estoc): void, que mostri la fitxa d'un producte.
- Crida-la tres vegades amb tres productes diferents, dins d'un .
- Formata el preu amb number_format($preu, 2, ',', '.').

Exercici 3

- Crida targeta() canviant l'ordre dels arguments a posta: posa el preu on va el nom.
- Anota què passa i explica per què en un comentari. Torna-ho a deixar bé.

Apartat 3 · return no és echo

Aquest és l'apartat més important de l'activitat. Una funció pot fer dues coses molt diferents: escriure a la pantalla amb echo, o tornar un valor amb return.

La diferència és que, amb echo, el valor ja s'ha escrit i no en pots fer res més. Amb return, qui crida la funció decideix què vol fer-ne: mostrar-lo, sumar-lo, comparar-lo o passar-lo a una altra funció.

Amb echo

```
function iva(float $b): void
{
    echo $b * 1.21;      // escriu i prou
}
```

Amb return

```
function iva(float $b): float
{
    return $b * 1.21;    // el valor es teu
}
```

Amb echo no pots...	Amb return pots...
Sumar el resultat	`$total += iva($preu);`
Formatar-lo	`formatPreu(iva($preu));`
Comparar-lo	`if (iva($preu) > 100) { ... }`

> **La regla:** una funció calcula i torna. Qui la crida decideix si ho mostra, ho suma o ho guarda. Reserva l'echo per a les funcions que només pinten HTML, com les de l'apartat 1.

Exercicis

Crea el fitxer apartat3.php i fes-hi aquests exercicis:

Exercici 1

- Escriu preuAmbIvaEcho(float $base): void, que mostri el preu amb IVA amb un echo.
- Escriu preuAmbIva(float $base): float, que el torni amb return.
- Escriu formatPreu(float $preu): string, que torni el preu amb dos decimals i la paraula EUR.

Exercici 2

- Amb un array de tres preus, calcula el total amb IVA sumant els resultats de preuAmbIva() dins d'un foreach.
- Mostra el total passant-lo per formatPreu().
- Mostra el preu més car amb IVA combinant max(), preuAmbIva() i formatPreu() en una sola línia.
- Intenta fer el mateix amb preuAmbIvaEcho() i explica en un comentari per què no pots.

Apartat 4 · Valors per defecte

Un paràmetre pot tenir un valor per defecte. Si qui crida la funció no el passa, s'usa aquest valor; si el passa, mana el seu.

```
function preuAmbIva(float $base, float $iva = 0.21): float
{
    return round($base * (1 + $iva), 2);
}
```

```
preuAmbIva(100);        // 121    fa servir el 0.21
preuAmbIva(100, 0.10);  // 110    fa servir el 0.10
```

L'ordre importa: els paràmetres amb valor per defecte han d'anar SEMPRE al final. Si en poses un al mig, PHP no sabria quin valor va a quin paràmetre i dona un avís.

Exercicis

Crea el fitxer apartat4.php i fes-hi aquests exercicis:

Exercici 1

- Afegeix a preuAmbIva() un segon paràmetre $iva amb valor per defecte 0.21.
- Crida-la tres vegades: sense el segon paràmetre, amb 0.10 i amb 0.04. Mostra els tres resultats.

Exercici 2

- Escriu aplicaDescompte(float $preu, int $percentatge = 10): float.
- Crida-la amb el descompte per defecte i amb un del 25 %.

Exercici 3

- Escriu benvinguda(string $nom, string $salutacio = 'Hola'): string.
- Prova de posar el paràmetre opcional el primer i anota l'avís que dona PHP. Torna-ho a deixar bé.

Apartat 5 · Els tipus i strict_types

Els tipus dels paràmetres i del retorn són una promesa: la funció diu què espera rebre i què tornarà. Amb declare(strict_types=1) a la primera línia, PHP fa complir aquesta promesa i avisa si algú la trenca.

Si crides	Sense strict_types	Amb strict_types
`area(3.0, 4.0)`	Funciona	Funciona
`area(3, 4)`	Funciona	Funciona: un int on s'espera un float no perd informació
`area('3', 4.0)`	Funciona, PHP converteix sol	TypeError: must be of type float, string given
`area((float) '3', 4.0)`	Funciona	Funciona: la conversió la fas tu i es veu

Un detall que confon: el que compta és el declare del fitxer que FA la crida, no el del fitxer on està escrita la funció.

Exercicis

Crea el fitxer apartat5.php i fes-hi aquests exercicis:

Exercici 1

- Escriu area(float $base, float $altura): float i esgotat(int $unitats): bool.
- Crida area() amb dos decimals i amb dos enters. Comprova que les dues funcionen.
- Crida area('3', 4.0) amb strict_types activat i anota l'error exacte que dona.
- Arregla-ho convertint el text amb (float) i comprova que ara sí que va.
- Explica en un comentari per què és millor que PHP t'avisi en lloc de convertir pel seu compte.

Per veure l'error sense que s'aturi la pàgina: posa la crida dins d'un bloc try/catch. No cal que entenguis com funciona encara; copia aquest patró.

```
try {
    echo area('3', 4.0);
} catch (TypeError $e) {
    echo 'Error de tipus: ' . $e->getMessage();
}
```

Apartat 6 · L'àmbit de les variables

Una variable creada fora d'una funció no entra sola dins de la funció. I una variable creada dins d'una funció mor quan la funció acaba. Això sorprèn molta gent que ve d'altres llenguatges.

```
$botiga = 'BitBazar';          // variable global

function mostra(): string
{
    return 'Som a ' . $botiga;  // NO la veu: Warning
}

// La manera bona: la dada entra per un parametre
function mostraBe(string $nom): string
{
    return "Som a $nom";
}
```

La porta d'entrada d'una funció són els paràmetres i la de sortida és el return. No hi ha cap altra manera recomanable de passar-hi informació.

La paraula static

Hi ha una excepció: una variable declarada amb static recorda el seu valor entre crides.

Variable local normal

```
function compta(): int
{
    $n = 0;
    $n++;
    return $n;
}

// Tres crides donen 1, 1, 1
```

Variable static

```
function compta(): int
{
    static $n = 0;
    $n++;
```

```
    return $n;
}

// Tres crides donen 1, 2, 3
```

Exercicis

Crea el fitxer apartat6.php i fes-hi aquests exercicis:

Exercici 1

- Crea una variable global $botiga i una funció que intenti fer-la servir sense rebre-la.
- Carrega la pàgina i mira el log d'errors. Enganxa en un comentari el warning exacte que hi apareix.
- Escriu la versió bona, que rep el nom per un paràmetre, i comprova que ara funciona.

Exercici 2

- Escriu comptaLocal(): int amb una variable normal i comptaStatic(): int amb una variable static.
- Crida cadascuna tres vegades seguides i mostra els resultats.
- Explica en un comentari la diferència entre les dues sortides.

No facis servir global: existeix una paraula clau global que deixa veure les variables de fora, però gairebé tothom recomana no utilitzar-la: fa que una funció depengui de coses que no es veuen a la crida. Els paràmetres són la manera correcta.

Apartat 7 · Funcions que treballen amb arrays

És el que faràs la major part del temps: una funció que rep un array de dades i en treu un valor, o que en torna un altre array ja filtrat.

Fes servir aquest catàleg a tots els exercicis d'aquest apartat i al repte final:

```
$productes = [
    ['nom' => 'Teclat mecanic RGB',   'categoria' => 'teclats',
     'preu' => 79.90,  'estoc' => 4],

    ['nom' => 'Ratoli vertical',      'categoria' => 'ratolins',
     'preu' => 29.95,  'estoc' => 7],

    ['nom' => 'Monitor 27" QHD',      'categoria' => 'monitors',
     'preu' => 289.00, 'estoc' => 0],

    ['nom' => 'Auriculars ANC',       'categoria' => 'audio',
     'preu' => 129.00, 'estoc' => 2],

    ['nom' => 'Hub USB-C 7 en 1',     'categoria' => 'accessoris',
     'preu' => 34.90,  'estoc' => 20],

    ['nom' => 'Suport per a portatil','categoria' => 'accessoris',
     'preu' => 24.50,  'estoc' => 0],
];
```

Dos models

Rep un array i torna un valor

```
function valorEstoc(array $productes): float
{
    $total = 0.0;
    foreach ($productes as $item) {
        $total += $item['preu'] * $item['estoc'];
    }
    return round($total, 2);
}
```

Rep un array i torna un array

```
function disponibles(array $productes): array
{
    $res = [];
    foreach ($productes as $item) {
        if ($item['estoc'] > 0) {
            $res[] = $item;
        }
    }
    return $res;
}
```

L'array entra per còpia: el que facis amb ell dins de la funció no toca l'array original de fora.

Exercicis

Crea el fitxer apartat7.php i fes-hi aquests exercicis:

Exercici 1: funcions que tornen un valor

- valorEstoc(array $productes): float — la suma de preu per unitats de tots els productes.
- quantsEsgotats(array $productes): int — quants productes tenen l'estoc a zero.
- preuMitja(array $productes): float — el preu mitjà, amb dos decimals.
- Fes que preuMitja() torni 0 si l'array és buit, en lloc de dividir entre zero. Prova-ho cridant-la amb [].

Exercici 2: funcions que tornen un array

- nomesDisponibles(array $productes): array — només els que tenen estoc.
- elMesCar(array $productes): array — el producte sencer amb el preu més alt.
- resumCataleg(array $productes): array — un array associatiu amb les claus total, disponibles i mes_car.
- Mostra el resum recorrent-lo amb un foreach, no escrivint les línies a mà.

Repte final · El catàleg de BitBazar

Una sola pàgina, repte.php, feta tota amb funcions. Fes servir el catàleg de l'apartat 7.

Funcions que ha de tenir

Funció	Què fa	Torna
`preuAmbIva()`	Calcula el preu amb IVA. El tipus d'IVA és un paràmetre opcional	float
`formatPreu()`	Formata un preu a l'estil català: 1.234,50 EUR	string
`estatEstoc()`	'Esgotat' si en queden 0, 'Últimes unitats' si en queden 3 o menys, i 'Disponible' en la resta de casos	string
`filtraCategoria()`	Els productes d'una categoria. Amb 'totes', el catàleg sencer	array
`resumCataleg()`	Array associatiu amb total, disponibles, esgotats i valor de l'estoc	array
`pintaTargeta()`	Escriu l'HTML de la targeta d'un producte	void
`pintaGraella()`	Escriu la graella sencera cridant pintaTargeta() per a cada producte	void

Què ha de mostrar la pàgina

1. El resum del catàleg, recorregut amb un foreach.
2. La graella amb tots els productes, cridant pintaGraella().
3. La graella amb només una categoria, cridant pintaGraella(filtraCategoria(...)).
4. Una tercera crida amb una categoria que no existeix. Ha de mostrar un missatge, no una graella buida ni un error.

> **La prova que ho has fet bé:** pintaGraella() ha de cridar pintaTargeta(), i pintaTargeta() ha de cridar formatPreu() i preuAmbIva(). Si ho has muntat així, canviar el format del preu es fa en un sol lloc i es nota a tota la pàgina.

> **Cap funció del repte pot repetir codi d'una altra.** Si veus el mateix càlcul escrit dues vegades en dues funcions diferents, has de treure'l a una tercera funció i cridar-la des de totes dues.

Com s'entrega

Comprimeix la carpeta M0613_ACT05 sencera en un fitxer ZIP i puja'l a Clickedu. Ha de contenir els set apartats, el repte i el full d'estils.

- Totes les funcions han de portar el seu bloc PHPDoc amb @param i @return. (Busca com fer-ho, ho mirarem a classe el pròxim dia).
- Tots els fitxers han de començar amb declare(strict_types=1);.
- Cap pàgina pot donar warnings al log, excepte el de l'apartat 6, que és a posta.
- Afegeix una captura de pantalla del repte funcionant dins d'una carpeta captures/.

I, com sempre, puja-ho també al teu repositori de Git:

```
git add .
```

```
git commit -m "M0613 ACT05: les funcions"
git push
```

Es corregeix executant-ho. Obriré cada apartat al navegador i miraré el log. Una funció que no es pot cridar, o que dona un warning, no compta encara que el codi sembli correcte.